

Reliability and Recovery of Amazon SQS Messaging under Injected Consumer and Downstream Failures

Anjaneya Reddy Gurram (Anji) | Student ID: 24288853

MSc Cloud Computing — National College of Ireland — Short Explanation Guide

Baseline: Kyrychenko et al. (2025) WSEAS — DOI 10.37394/23202.2025.24.4

Read aloud in simple English. Numbers come from STATUS.md / results — do not invent live AWS metrics.

1. What is the project?

This project studies Amazon SQS — a cloud queue between producer and consumer — when things go wrong. Prior work tunes SQS for speed when consumers are healthy. I ask: if the consumer dies or the database rejects writes, do those “optimal” settings still keep messages safe?

Artefact: SQS Standard + Dead-Letter Queue → Lambda consumer → DynamoDB, with an SSM fault switch. A local simulator runs the full matrix for free. Live lite confirmation ran in eu-west-1. Stack destroyed after initial_eval_1.

Pitch: “I test how Amazon SQS settings like visibility timeout and retry limits affect message safety and recovery when consumers or downstream stores fail.”

2. Problem statement + real-life example

SQS Standard is at-least-once: a message can arrive more than once. Wrong visibility timeout or maxReceiveCount causes loss, duplicates, or endless retries that burn Lambda money. Throughput-only papers can recommend long VT that looks great when healthy but recovers slowly after a crash.

Real-life example — café pager. Visibility timeout = how long a ticket stays taken before returning if the waiter disappears. maxReceiveCount / DLQ = after N failed tries, put the ticket in the manager’s problem drawer. Batch size = grabbing five tickets at once. Kyrychenko times waiters when nobody drops trays; I study fainting waiters and kitchen rejects.

Industry angle: SQS appears in order intake, image pipelines, notifications. Copying a throughput optimum into a failure-prone path without measuring loss and recovery is risky.

3. Baseline paper

Kyrychenko, Ostapov and Kyrychenko (2025), WSEAS Transactions on Systems (DOI 10.37394/23202.2025.24.4). SQS-backed serverless batch pipeline; M/M/1 and M/M/k models; 500,000 records; varied batch size, visibility timeout, delivery delay. Reported optimum around batch 50, VT 600 s, delay 300 s.

Gap: emphasises throughput/latency/cost under steady-state healthy consumers. Reliability under injected consumer or downstream failure is not measured with message-level loss, duplicates, DLQ capture, and recovery time. That is this thesis niche.

4. How we solve it / research question

RQ: How do SQS knobs (visibility timeout, maxReceiveCount / DLQ redrive) affect message loss, duplicates, DLQ capture, and recovery under injected consumer and downstream failures?

Solution: hold async architecture fixed; inject consumer_kill / unhandled_error / datastore reject-timeout; vary VT and MRC; measure loss, dup, DLQ, recovery, plus throughput for baseline comparability.

Authoritative H1–H3 stats stay in localsim. Live lite cells are smoke (n=1, ≈200 orders). Do not pretend live n=1 is Holm-powered.

STATUS examples: consumer_kill VT=30 MRC=5 → loss 0, dup 0.025, recovery ≈3.1 s. Unhandled_error MRC=1 → DLQ 0.23, success 0.805. Must NOT claim: DIVE/adaptive VT evaluated; localsim is perfect AWS twin; results transfer to Kafka parameter-for-parameter.

5. Dataset / data

Fully synthetic orders. No real customers. No PII. Own AWS account.

- Localsim: full factorial for confirmatory H1–H3 (results/summary/stats_H1_H2_H3.json).
- Live initial eval: configs/live_key_cells.yaml — 4 cells × n=1, ≈200 orders/cell, eu-west-1, stack sqs-rr-dev, ESM max_concurrency=2.
- Faults injected (consumer kill, unhandled error) — not from production incidents.
- Metrics: loss, duplicate, DLQ capture, recovery seconds, throughput, success.
- Destroy verified: 0 TF resources; Lambda/SQS/DynamoDB absent.

6. Implementation + technologies / AWS services

- Amazon SQS Standard — main order queue.
- SQS Dead-Letter Queue — after maxReceiveCount failures.
- AWS Lambda — consumer via Event Source Mapping.
- DynamoDB — processed order state.
- SSM Parameter Store — fault switches.
- SAM + Terraform — deploy/destroy sqs-rr-dev.
- Python experiment runner + local simulator.
- CloudWatch — invocation/metrics cross-checks.

7. End-to-end flow

- Generate synthetic orders; publish to SQS.
- Lambda receives batch; visibility timeout starts.
- Optionally inject fault (kill / error / store reject-timeout).
- On failure: message reappears after VT, up to MRC times.
- After MRC exhausted → DLQ (capture measured).
- On success: write DynamoDB; record latency/throughput.
- Runner writes manifest + loss/dup/DLQ/recovery metrics.
- Destroy stack; keep results/live/initial_eval_1/.

8. Exact speaking script (dataset + implementation + technologies)

“My data is fully synthetic order messages — no real users. The big statistical matrix runs in a local simulator so I can afford confirmatory tests. On AWS I only ran a lite four-cell confirmation: about two hundred orders per cell in eu-west-1, then I destroyed the stack.”

“I implemented an SQS Standard queue with a dead-letter queue, a Lambda consumer, DynamoDB for state, and an SSM switch to inject consumer and downstream faults. I vary visibility timeout and maxReceiveCount. Technologies are Python, SAM, Terraform, and the AWS SQS, Lambda, and DynamoDB APIs. I measure loss, duplicates, DLQ capture, and recovery time — not just healthy throughput.”

9. STATUS snapshot + five Q&A

CA2 / gate	ALIGNMENT=100; INITIAL_EVAL_PASS=yes
Live	4 lite cells; ≈200 orders; eu-west-1
Stats home	localsim H1–H3 (live = smoke)
Destroy	DESTROY_CONFIRMED=yes

Q: Same as Kyrychenko?

A: No — they optimise healthy throughput; I inject failures and measure loss/dup/DLQ/recovery.

Q: Real customer data?

A: No — synthetic orders on my own AWS account.

Q: Are live n=1 cells your main stats?

A: No — live is smoke; H1–H3 confirmatory stats are localsim.

Q: What knobs do you vary?

A: Visibility timeout and maxReceiveCount / DLQ redrive under injected faults.

Q: Is the AWS stack still up?

A: No — DESTROY_CONFIRMED=yes after initial_eval_1.